S.G.R.S.I

Sistema de Gestión de Recursos y Soporte de Informática

Programacion

EnterpriceDeKlan

Rol	Apellido	Nombre	documento identidad	email
Coordinador	Gonzaléz	Julian	5.742892-8	julian.gonzalez.arrascaeta@gmail.com
Sub-Coordinador	Cuello	Santiago	5731237-3	augustocrafter22@gmail.com
Integrante 1	Forcelledo	Ezequiel	5724706-7	ezequielforcelledo9@gmail.com
Integrante 2	Venis	Federico	6486057-3	fede.venis@gmail.com

Docente: Lopez, Leandro

PRIMERA ENTREGA

Fecha de entrega:

23 oct 2026

CETP

ÍNDICE

Herramientas y tecnologías **4**

IDE/Editor de Código 4

Estandares de codificacion 4

Convenciones de commit y resolución de issues 5

Versionado 6

Estructura de archivos y directorios del sistema 6

Persistencia **8**

Participantes (nivel de título 2) 8

Normalización **9**

Patrones de diseño **17**

Acceso al repositorio 20

Anexos **21**

Bibliografía utilizada **24**

Herramientas de investigación **25**

Herramientas y tecnologías

A continuación, se detallan las herramientas y las tecnologías seleccionadas por el equipo para el desarrollo del software, junto con su respectiva justificación en cada eleccion.

Tabla de herramientas utilizadas

Herramienta o Tecnología	Uso en el proyecto	Por qué lo elegimos
Visual Studio Code 1.140.0 windows.	Lo utilizamos para escribir todo el código. (PHP, JS, HTML, CSS, SQL)	Es el editor que nos recomendó usar el profesor. Tiene integración por defecto con git y github, soporta los lenguajes que usamos y tiene extensiones útiles para el trabajo.
Git 2.53.0.windows.2	Fue usado para controlar las versiones en local, con commits, ramas y etiquetas	Es lo estándar, permite usar ramas para trabajar por issue sin dañar el trabajo del otro.
Github	Usamos el repositorio colaborativo, issues y las pull request.	Lo sugirió el docente, además de ofrecer repositorios privados gratuitos y la capacidad de cerrar issues con commits.
PHP 8.2.12	Lo usamos para el servidor: Controladores, lógica, validaciones y acceso a datos	Es un lenguaje para servidor que vimos en el curso, tiene sesiones y manejo de excepciones. Además de que es compatible con otras dependencias del proyecto.
XAMPP 8.2.12	Utilizado para su entorno que incluye Apache, PHP y MariaDB.	Es gratuito e intuitivo. El entorno se levanta solo sin configuraciones manuales.

Apache 2.4.58	Servidor web que ejecuta el PHP y hace funcionar todas las páginas guardadas en la carpeta /public	Viene integrado en el XAMPP.
MariaDB 10.4.32	Motor de la base de datos	Es gratis y altamente compatible con MySQL. También viene integrado en XAMPP.
PDO (Extensión de PHP)	Conexión con la base de datos y consultas.	Da la capacidad de usar consultas preparadas con parámetros, lo que ayuda a cumplir el requisito de seguridad contra inyecciones SQL.
Composer 2.10.2	Usado para gestionar las dependencias de PHP	Lo usamos para instalar las dependencias que necesitábamos para PHP.
vlucas/phpdotenv v5.6.4	Lo tenemos para guardar y cargar las credenciales de la base de datos desde .env	Usado exclusivamente por temas de seguridad. Para evitar subir las contraseñas al repositorio.
JavaScript, HTML5 y CSS3	Usado para mostrar páginas, darle estilo y hacer llamadas a la api con fetch	Cubre todo lo que vimos en el curso. Mantiene el código simple y fácil de entender para los integrantes.
Figma	Elegido para crear prototipos visuales de las pantallas	Debido a la simpleza y uso previo, optamos por la página Figma para el diseño.

Justificación de librerías con ejemplo

phpdotenv: en config/config.php se hacemos Dotenv\Dotenv::createImmutable(RUTA_RAIZ)->load(). Guardamos las credenciales en .env, que no se sube. En el repositorio solo está .env.example con BD_HOST, BD_PORT, BD_USER, BD_PASS y BD_NAME. Esto sirve para mostrar que la configuración sensible queda fuera del código.

PDO con consultas preparadas: en DAOEquipo::listar el filtro se arma con parámetros (e.estado = :estado) y se ejecuta con prepare() y execute($parametros). Así el dato del usuario no queda concatenado en el SQL.

fetch: en public/assets/js/nucleo/api.js, llamarApi(ruta, {metodo, parametros, cuerpo}) Centralizamos toda la comunicación con el servidor. Ahí también se agrega el token CSRF únicamente en todo método que toque y gestione los datos.

Estándares de codificación.

Para los estándares de codificación, decidimos utilizar un reglamento basado en el libro Codigo limpio - Robert Cecil Martin.

Nombres:

Regla 1: Las clases son sustantivos y los métodos son verbos. EJ: adentro de DAOEquipo tiene listar, obtener, crear.

Regla 2: Las clases se nombran con un concepto y una palabra. EJ: Controlador [“de que” el recurso].

Regla 3:

- Los archivos PHP se nombran con PascalCase.
- Variables y métodos con camelCase y en español.
- Tablas en mayúsculas y columnas en camelCase.
- Prefijos restringidos a pk_ o fk_
- Archivos JS y CSS en camelCase.

Comentarios:

Regla 1: Todo método público lleva PHPDoc con @param, @return y @throws.

Regla 2: Los comentarios en línea explican el porqué. El código comentado queda prohibido.

Manejo de errores:

Regla 1: Los controladores lanzan excepciones con códigos HTTP.

Regla 2: index.php captura las excepciones y las entrega en ManejadorErrores.

Regla 3: Las operaciones de varios pasos usan rollBack.

Repositorio

Github es una plataforma de repositorios Git en la nube. Es gratuita y fue la seleccionada debido que:

- Los repositorios privados con control de invitación.
- Issues para registrar requerimientos.
- Pull Request para revisar e integrar ramas.
- Cierre de issues usando keywords

- Historial de commits visible
- Integración nativa con VS Code.

Ventajas

- Es el repositorio que usamos y nos sugirió el docente. Fácil seguimiento y corrección.
- Nos permitió usar ramas para dividir requerimientos por issue y vincularlos.
- La interfaz es simple y es muy fácil de usar.
- Mucha documentación al alcance de la mano.

Desventajas

- Depende de servicios externos. Sin conexión no se puede consultar issues ni las pull request.
- En nuestro caso nuestro repositorio depende de una cuenta personal y no a una organización. En caso de problemas con la cuenta nuestro equipo puede perder el control del trabajo.

Justificación

Lo elegimos porque cumple con los requisitos del docente, soporta flujo de ramas, tiene una curva de aprendizaje baja y se integra perfectamente con el editor que elegimos.

Gestión de ramas

La estrategia de ramas adoptada por el equipo es una variante simplificada de GitHub Flow, donde cada rama se corresponde con un issue (requerimiento) puntual del sistema.

La rama “main” es la estable y entregable, solo recibe código con Pull Request. Con la excepción de la documentación que se admite directamente .

Todas las ramas parten de la rama “main” como indica GitHub Flow.

Las ramas se crean cuando se toma un issue para trabajarlo. Primero se crea la rama y luego se trabaja el código. En caso de cambios pequeños se usan ramas temporales.

El merge se realiza desde Pull Request en GitHub apuntando hacia main

Convenciones de commit y resolución de issues

Prefijos utilizados

- feat: para nuevas funcionalidades agregadas al sistema.
- fix: para correcciones de errores sobre funcionalidad existente.

- docs: para cambios relacionados exclusivamente con documentación (únicos commits admitidos directamente sobre main).

Closing keywords

- Se utilizan palabras clave de cierre de GitHub (Closes #, Fixes #, Resolves #) en la descripción del commit o del Pull Request, de forma que al mergear la rama a main el issue correspondiente se cierra automáticamente.

Normas para la descripción de los commits

- El mensaje comienza con el prefijo correspondiente (feat/fix/docs), seguido de una descripción breve en modo imperativo de la tarea realizada.

Cuando usar un commit

- Cuando el cambio realizado sea algo significativo y el sistema funcione.

Cuando cerrar un issue

- Cuando su rama se integra al main a través de Pull Request.

Cómo agrupar cambios y hacer el mensaje

- El commit agrupa los cambios que resuelven una misma cosa. No se mezclan en un mismo commit funcionalidades con correcciones ajenas al issue. El mensaje empieza con uno de los prefijos y si hace falta explicar porqué, va en la descripción del commit y no en el título. Cada commit debe entenderse sin abrir el código.

Versionado

El equipo utiliza Versionado Semántico (Semantic Versioning) bajo el formato Mayor.Menor.Parche (0.0.0):

Versión Mayor: se incrementa únicamente cuando la primera entrega del proyecto está lista, marcando el paso del proyecto a una fase entregable.

Versión Menor: se incrementa cada vez que se agrega una nueva funcionalidad o módulo al proyecto, siempre que se mantenga la compatibilidad con lo ya implementado.

Versión Parche: se incrementa exclusivamente ante correcciones o cambios ligeros que no modifican la funcionalidad existente.

Estructura visual:

baseDeDatos

docs

PROYECTO

- app
  - controlador
  - i18n
  - modelo
  - núcleo
  - rutas
  - vista
- config
- public
  - assets
    - css
    - img
    - js
  - paginas
- vendor
  - composer
  - graham-campbell
  - phpoption
  - symfony
  - vlucas
- .env

.env.example

.gitattributes

.gitignore

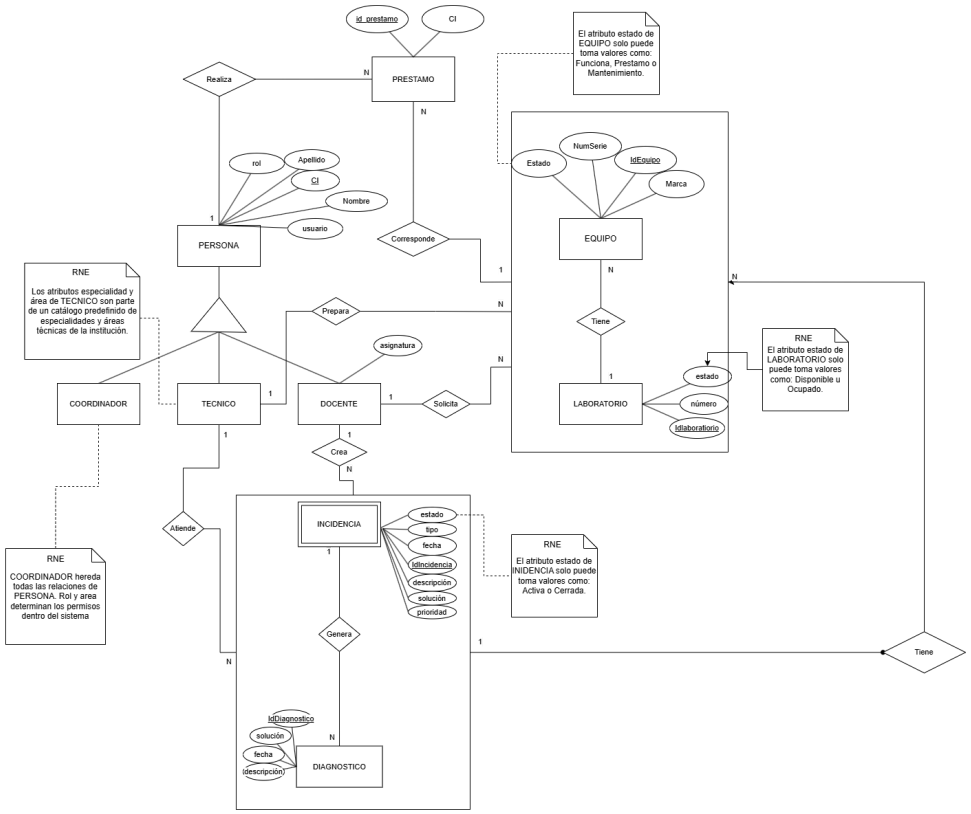
README.md

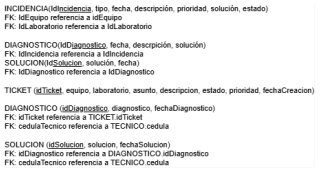
repositorio.txt

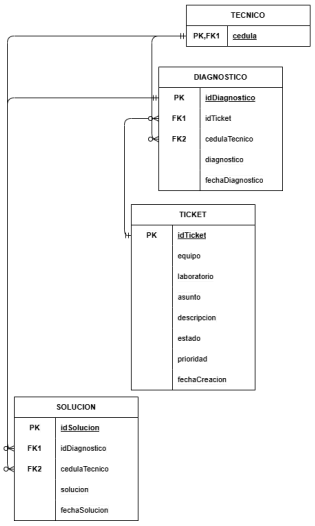
Persistencia

Participantes (nivel de título 2)

El sistema utiliza una base de datos de lenguaje MySQL y controla los datos del programa a través de php. Aquí el diagrama:







Normalización

Relación EQUIPO

EQUIPO (idEquipo, idLaboratorio, marca, estado, disponibilidad, informacion)

Clave Primaria: {idEquipo}

Primera Forma Normal

Se encuentra en 1FN porque todos sus atributos son atómicos.

Segunda Forma Normal

Clave de un solo atributo ({idEquipo}), por lo tanto no puede existir dependencia parcial. Se analiza:

{idEquipo} -> {idLaboratorio} D.F. Total

{idEquipo} -> {marca} D.F. Total

{idEquipo} -> {estado} D.F. Total

{idEquipo} -> {disponibilidad} D.F. Total

{idEquipo} -> {informacion} D.F. Total

La relación se encuentra en 2FN porque todos los atributos no primos dependen totalmente de la clave primaria.

Tercera Forma Normal

Se analiza si algún atributo no primo determina a otro:

{marca} -> {estado}? No existe tal dependencia (equipos de la misma marca pueden estar en distinto estado)

{idLaboratorio} -> {marca}? No existe (un laboratorio tiene equipos de varias marcas)

{idLaboratorio} -> {estado}? No existe

{estado} -> {disponibilidad}? No existe tal dependencia. Aunque en la práctica un equipo "Funcionando" suele estar "Disponible", la disponibilidad también depende de los préstamos: un equipo "Funcionando" pasa a "No disponible" mientras tiene un préstamo activo. Por eso el estado no determina la disponibilidad.

{disponibilidad} -> {estado}? No existe (un equipo "No disponible" puede estar funcionando, en mantenimiento o dañado)

{marca} -> {informacion}? No existe (informacion es texto libre propio de cada equipo)

Ningún atributo no primo determina a otro atributo no primo. La relación se encuentra en 3FN.

Resultado

EQUIPO (idEquipo, idLaboratorio, marca, estado, disponibilidad, informacion)

PK: idEquipo

idLaboratorio es FK de LABORATORIO(idLaboratorio)

Relación PRESTAMO

PRESTAMO (idPrestamo, idEquipo, cedulaSolicitante, fechaPrestamo, fechaDevolucionEstimada, fechaDevolucionReal, estado)

Clave Primaria: {idPrestamo}

Primera Forma Normal

Se encuentra en 1FN porque todos sus atributos son atómicos.

Segunda Forma Normal

Clave de un solo atributo ({idPrestamo}), por lo tanto no puede existir dependencia parcial. Se analiza:

{idPrestamo} -> {idEquipo} D.F. Total

{idPrestamo} -> {cedulaSolicitante} D.F. Total

{idPrestamo} -> {fechaPrestamo} D.F. Total

{idPrestamo} -> {fechaDevolucionEstimada} D.F. Total

{idPrestamo} -> {fechaDevolucionReal} D.F. Total

{idPrestamo} -> {estado} D.F. Total

La relación se encuentra en 2FN porque todos los atributos no primos dependen totalmente de la clave primaria.

Tercera Forma Normal

Se analiza si algún atributo no primo determina a otro:

{idEquipo} -> {cedulaSolicitante}? No existe (un mismo equipo se presta a distintas personas a lo largo del tiempo)

{cedulaSolicitante} -> {idEquipo}? No existe (una persona puede pedir distintos equipos)

{fechaPrestamo} -> {fechaDevolucionEstimada}? No existe (la fecha estimada se ingresa al registrar el préstamo, no se calcula a partir de la fecha de inicio)

{fechaDevolucionReal} -> {estado}? Existe una relación: el estado es "Devuelto" cuando hay fecha real de devolución y "Activo" cuando no la hay. Es un atributo derivado que se conserva de forma deliberada para filtrar préstamos activos de manera directa. Para evitar contradicciones, ambos valores se modifican en una misma sentencia UPDATE al registrar la devolución.

Salvo el atributo derivado indicado, ningún atributo no primo determina a otro. Se documenta como redundancia controlada.

Resultado

PRESTAMO (idPrestamo, idEquipo, cedulaSolicitante, fechaPrestamo, fechaDevolucionEstimada, fechaDevolucionReal, estado)

PK: idPrestamo

idEquipo es FK de EQUIPO(idEquipo)

cedulaSolicitante es FK de USUARIO(cedula)

Relación USUARIO

USUARIO (cedula, nombre, apellido, clave, activo)

Clave Primaria: {cedula}

Primera Forma Normal

Se encuentra en 1FN porque todos sus atributos son atómicos.

Segunda Forma Normal

Clave de un solo atributo ({cedula}), por lo tanto no puede existir dependencia parcial. Se analiza:

{cedula} -> {nombre} D.F. Total

{cedula} -> {apellido} D.F. Total

{cedula} -> {clave} D.F. Total

{cedula} -> {activo} D.F. Total

La relación se encuentra en 2FN porque todos los atributos no primos dependen totalmente de la clave primaria.

Tercera Forma Normal

Se analiza si algún atributo no primo determina a otro:

{nombre} -> {apellido}? No existe (personas distintas pueden compartir nombre)

{apellido} -> {nombre}? No existe (personas distintas pueden compartir apellido)

{activo} -> {nombre}? No existe (activo solo indica si la cuenta está habilitada)

Ningún atributo no primo determina a otro atributo no primo. La relación se encuentra en 3FN.

Resultado

USUARIO (cedula, nombre, apellido, clave, activo)

PK: cedula

Relación ADMINISTRADOR

ADMINISTRADOR (cedula)

Clave Primaria: {cedula}

Primera Forma Normal

Se encuentra en 1FN porque su único atributo es atómico.

Segunda Forma Normal

No hay atributos no primos que analizar.

Tercera Forma Normal

No hay atributos que analizar.

Resultado

ADMINISTRADOR (cedula)

PK: cedula

cedula es FK de USUARIO(cedula)

Relación TECNICO

TECNICO (cedula)

Clave Primaria: {cedula}

Primera Forma Normal

Se encuentra en 1FN porque su único atributo es atómico.

Segunda Forma Normal

No hay atributos no primos que analizar.

Tercera Forma Normal

No hay atributos que analizar.

Resultado

TECNICO (cedula)

PK: cedula

cedula es FK de USUARIO(cedula)

Relación DOCENTE

DOCENTE (cedula)

Clave Primaria: {cedula}

Primera Forma Normal

Se encuentra en 1FN porque su único atributo es atómico.

Segunda Forma Normal

No hay atributos no primos que analizar.

Tercera Forma Normal

No hay atributos que analizar.

Resultado

DOCENTE (cedula)

PK: cedula

cedula es FK de USUARIO(cedula)

Relación LABORATORIO

LABORATORIO (idLaboratorio, numeroLaboratorio, estado)

Clave Primaria: {idLaboratorio}

Primera Forma Normal

Se encuentra en 1FN porque todos sus atributos son atómicos.

Segunda Forma Normal

Clave de un solo atributo ({idLaboratorio}), por lo tanto no puede existir dependencia parcial.
Se analiza:

{idLaboratorio} -> {numeroLaboratorio} D.F. Total

{idLaboratorio} -> {estado} D.F. Total

La relación se encuentra en 2FN porque todos los atributos no primos dependen totalmente de la clave primaria.

Tercera Forma Normal

Se analiza si algún atributo no primo determina a otro:

{numeroLaboratorio} -> {estado}? No existe tal dependencia (el estado es una condición que cambia en el tiempo para cada laboratorio, no se deriva de su número)

{estado} -> {numeroLaboratorio}? No existe (varios laboratorios comparten el mismo estado)

Ningún atributo no primo determina a otro atributo no primo. La relación se encuentra en 3FN.

Resultado

LABORATORIO (idLaboratorio, numeroLaboratorio, estado)

PK: idLaboratorio

Relación SOLICITUD_LABORATORIO

SOLICITUD_LABORATORIO (idSolicitud, idLaboratorio, cedulaSolicitante, solicitaSoftware, detalle, restricciones, fechaEstimada, horaEstimada, fechaCreacion)

Clave Primaria: {idSolicitud}

Primera Forma Normal

Se encuentra en 1FN porque todos sus atributos son atómicos.

Segunda Forma Normal

Clave de un solo atributo ({idSolicitud}), por lo tanto no puede existir dependencia parcial. Se analiza:

{idSolicitud} -> {idLaboratorio} D.F. Total

{idSolicitud} -> {cedulaSolicitante} D.F. Total

{idSolicitud} -> {solicitaSoftware} D.F. Total

{idSolicitud} -> {detalle} D.F. Total

{idSolicitud} -> {restricciones} D.F. Total

{idSolicitud} -> {fechaEstimada} D.F. Total

{idSolicitud} -> {horaEstimada} D.F. Total

{idSolicitud} -> {fechaCreacion} D.F. Total

La relación se encuentra en 2FN porque todos los atributos no primos dependen totalmente de la clave primaria.

Tercera Forma Normal

Se analiza si algún atributo no primo determina a otro:

{idLaboratorio} -> {cedulaSolicitante}? No existe (un laboratorio es solicitado por distintos docentes)

{cedulaSolicitante} -> {idLaboratorio}? No existe (un docente solicita distintos laboratorios)

{solicitaSoftware} -> {detalle}? No existe (el detalle es texto libre y puede existir sin pedido de software)

{fechaEstimada} -> {horaEstimada}? No existe (una misma fecha admite distintas horas)

{fechaCreacion} -> {fechaEstimada}? No existe (la fecha de uso la elige el docente, no se calcula desde la creación)

Ningún atributo no primo determina a otro atributo no primo. La relación se encuentra en 3FN.

Resultado

SOLICITUD_LABORATORIO (idSolicitud, idLaboratorio, cedulaSolicitante, solicitaSoftware, detalle, restricciones, fechaEstimada, horaEstimada, fechaCreacion)

PK: idSolicitud

idLaboratorio es FK de LABORATORIO(idLaboratorio)

cedulaSolicitante es FK de DOCENTE(cedula)

Relación TICKET

TICKET (idTicket, laboratorio, equipo, asunto, descripcion, turno, grupo, profesor, estado, prioridad, fechaCreacion, fechaFinalizacion)

Clave Primaria: {idTicket}

Primera Forma Normal

Se encuentra en 1FN porque todos sus atributos son atómicos.

Segunda Forma Normal

Clave de un solo atributo ({idTicket}), por lo tanto no puede existir dependencia parcial. Se analiza:

{idTicket} -> {laboratorio} D.F. Total

{idTicket} -> {equipo} D.F. Total

{idTicket} -> {asunto} D.F. Total

{idTicket} -> {descripcion} D.F. Total

{idTicket} -> {turno} D.F. Total

{idTicket} -> {grupo} D.F. Total

{idTicket} -> {profesor} D.F. Total

{idTicket} -> {estado} D.F. Total

{idTicket} -> {prioridad} D.F. Total

{idTicket} -> {fechaCreacion} D.F. Total

{idTicket} -> {fechaFinalizacion} D.F. Total

La relación se encuentra en 2FN porque todos los atributos no primos dependen totalmente de la clave primaria.

Tercera Forma Normal

Se analiza si algún atributo no primo determina a otro:

{equipo} -> {laboratorio}? No existe dentro de esta relación. Un equipo puede cambiar de laboratorio (queda registrado en MOVIMIENTO_EQUIPO) y el ticket debe recordar en qué laboratorio ocurrió la incidencia aunque el equipo se mueva después. Al crear el ticket, el sistema verifica que el equipo esté en ese laboratorio.

{laboratorio} -> {equipo}? No existe (un laboratorio tiene muchos equipos)

{grupo} -> {profesor}? No existe (un mismo grupo puede tener distintos profesores)

{profesor} -> {grupo}? No existe (un profesor dicta a distintos grupos)

{turno} -> {grupo}? No existe (en un mismo turno hay varios grupos)

{estado} -> {prioridad}? No existe (la prioridad se asigna caso a caso, independiente del estado)

{estado} -> {fechaFinalizacion}? No existe (la fecha corresponde al momento en que se finalizó cada ticket y no queda fijada por el estado)

{asunto} -> {descripcion}? No existe (la descripción es texto libre propio de cada ticket)

Ningún atributo no primo determina a otro atributo no primo. La relación se encuentra en 3FN.

Resultado

TICKET (idTicket, laboratorio, equipo, asunto, descripcion, turno, grupo, profesor, estado, prioridad, fechaCreacion, fechaFinalizacion)

PK: idTicket

laboratorio es FK de LABORATORIO(idLaboratorio)

equipo es FK de EQUIPO(idEquipo)

Relación DIAGNOSTICO

DIAGNOSTICO (idDiagnostico, idTicket, cedulaTecnico, diagnostico, fechaDiagnostico)

Clave Primaria: {idDiagnostico}

Primera Forma Normal

Se encuentra en 1FN porque todos sus atributos son atómicos.

Segunda Forma Normal

Clave de un solo atributo ({idDiagnostico}), por lo tanto no puede existir dependencia parcial. Se analiza:

{idDiagnostico} -> {idTicket} D.F. Total

{idDiagnostico} -> {cedulaTecnico} D.F. Total

{idDiagnostico} -> {diagnostico} D.F. Total

{idDiagnostico} -> {fechaDiagnostico} D.F. Total

La relación se encuentra en 2FN porque todos los atributos no primos dependen totalmente de la clave primaria.

Tercera Forma Normal

Se analiza si algún atributo no primo determina a otro:

{idTicket} -> {cedulaTecnico}? No existe (el modelo permite varios diagnósticos por ticket, hechos por distintos técnicos)

{cedulaTecnico} -> {idTicket}? No existe (un técnico diagnostica muchos tickets)

{diagnostico} -> {idTicket}? No existe (el diagnóstico es texto libre y puede repetirse en tickets distintos)

{fechaDiagnostico} -> {idTicket}? No existe (en una misma fecha se diagnostican varios tickets)

Ningún atributo no primo determina a otro atributo no primo. La relación se encuentra en 3FN.

Resultado

DIAGNOSTICO (idDiagnostico, idTicket, cedulaTecnico, diagnostico, fechaDiagnostico)

PK: idDiagnostico

idTicket es FK de TICKET(idTicket)

cedulaTecnico es FK de TECNICO(cedula)

Relación SOLUCION

SOLUCION (idSolucion, idDiagnostico, cedulaTecnico, solucion, fechaSolucion)

Clave Primaria: {idSolucion}

Primera Forma Normal

Se encuentra en 1FN porque todos sus atributos son atómicos.

Segunda Forma Normal

Clave de un solo atributo ({idSolucion}), por lo tanto no puede existir dependencia parcial. Se analiza:

{idSolucion} -> {idDiagnostico} D.F. Total

{idSolucion} -> {cedulaTecnico} D.F. Total

{idSolucion} -> {solucion} D.F. Total

{idSolucion} -> {fechaSolucion} D.F. Total

La relación se encuentra en 2FN porque todos los atributos no primos dependen totalmente de la clave primaria.

Tercera Forma Normal

Se analiza si algún atributo no primo determina a otro:

{idDiagnostico} -> {cedulaTecnico}? No existe (el técnico que da la solución puede ser distinto del que hizo el diagnóstico)

{cedulaTecnico} -> {idDiagnostico}? No existe (un técnico registra soluciones de muchos diagnósticos)

{solucion} -> {idDiagnostico}? No existe (la solución es texto libre)

{fechaSolucion} -> {idDiagnostico}? No existe (en una misma fecha se registran varias soluciones)

Ningún atributo no primo determina a otro atributo no primo. La relación se encuentra en 3FN.

Resultado

SOLUCION (idSolucion, idDiagnostico, cedulaTecnico, solucion, fechaSolucion)

PK: idSolucion

idDiagnostico es FK de DIAGNOSTICO(idDiagnostico)

cedulaTecnico es FK de TECNICO(cedula)

Relación REPARACION

Modelo inicial:

REPARACION (idReparacion, idDiagnostico, idTicket, idEquipo, cedulaTecnico, reparacion, fechaReparacion)

Clave Primaria: {idReparacion}

Primera Forma Normal

Se encuentra en 1FN porque todos sus atributos son atómicos.

Segunda Forma Normal

Clave de un solo atributo ({idReparacion}), por lo tanto no puede existir dependencia parcial. Todos los atributos dependen de la clave:

{idReparacion} -> {idDiagnostico}, {idTicket}, {idEquipo}, {cedulaTecnico}, {reparacion}, {fechaReparacion} D.F. Total

La relación se encuentra en 2FN.

Tercera Forma Normal

Se analiza si algún atributo no primo determina a otro:

{idDiagnostico} -> {idTicket}? Existe: cada diagnóstico pertenece a un único ticket (por DIAGNOSTICO)

{idTicket} -> {idEquipo}? Existe: cada ticket se refiere a un único equipo (por TICKET)

Por lo tanto {idReparacion} -> {idDiagnostico} -> {idTicket} -> {idEquipo} es una dependencia transitiva. La relación NO se encuentra en 3FN: el mismo dato quedaba guardado dos veces y podía contradecirse (por ejemplo, una reparación con un equipo distinto al del ticket de su diagnóstico).

Se eliminan idTicket e idEquipo de REPARACION. Ambos se obtienen con REPARACION ⋈ DIAGNOSTICO ⋈ TICKET.

Se analiza nuevamente la relación resultante:

{idDiagnostico} -> {cedulaTecnico}? No existe (el técnico que repara puede ser distinto del que diagnosticó)

{cedulaTecnico} -> {idDiagnostico}? No existe (un técnico realiza muchas reparaciones)

{reparacion} -> {idDiagnostico}? No existe (la reparación es texto libre)

Ningún atributo no primo determina a otro atributo no primo. La relación se encuentra en 3FN.

Resultado

REPARACION (idReparacion, idDiagnostico, cedulaTecnico, reparacion, fechaReparacion)

PK: idReparacion

idDiagnostico es FK de DIAGNOSTICO(idDiagnostico)

cedulaTecnico es FK de TECNICO(cedula)

Relación HISTORIAL_ESTADO_EQUIPO

HISTORIAL_ESTADO_EQUIPO (idHistorialEstado, idEquipo, estadoAnterior, estadoNuevo, cedulaUsuario, fechaCambio)

Clave Primaria: {idHistorialEstado}

Primera Forma Normal

Se encuentra en 1FN porque todos sus atributos son atómicos.

Segunda Forma Normal

Clave de un solo atributo ({idHistorialEstado}), por lo tanto no puede existir dependencia parcial. Se analiza:

{idHistorialEstado} -> {idEquipo} D.F. Total

{idHistorialEstado} -> {estadoAnterior} D.F. Total

{idHistorialEstado} -> {estadoNuevo} D.F. Total

{idHistorialEstado} -> {cedulaUsuario} D.F. Total

{idHistorialEstado} -> {fechaCambio} D.F. Total

La relación se encuentra en 2FN porque todos los atributos no primos dependen totalmente de la clave primaria.

Tercera Forma Normal

Se analiza si algún atributo no primo determina a otro:

{estadoAnterior} -> {estadoNuevo}? No existe (desde un mismo estado se puede pasar a distintos estados)

{estadoNuevo} -> {estadoAnterior}? No existe (a un mismo estado se llega desde distintos estados)

{idEquipo} -> {estadoAnterior}? No existe (cada equipo cambia de estado varias veces, y cada fila es un hecho único)

{cedulaUsuario} -> {idEquipo}? No existe (un usuario modifica el estado de varios equipos)

Ningún atributo no primo determina a otro atributo no primo. La relación se encuentra en 3FN.

Resultado

HISTORIAL_ESTADO_EQUIPO (idHistorialEstado, idEquipo, estadoAnterior, estadoNuevo, cedulaUsuario, fechaCambio)

PK: idHistorialEstado

idEquipo es FK de EQUIPO(idEquipo)

cedulaUsuario es FK de USUARIO(cedula)

Relación MOVIMIENTO_EQUIPO

MOVIMIENTO_EQUIPO (idMovimiento, idEquipo, idLaboratorioAnterior, idLaboratorioNuevo, cedulaUsuario, fechaMovimiento)

Clave Primaria: {idMovimiento}

Primera Forma Normal

Se encuentra en 1FN porque todos sus atributos son atómicos.

Segunda Forma Normal

Clave de un solo atributo ({idMovimiento}), por lo tanto no puede existir dependencia parcial. Se analiza:

{idMovimiento} -> {idEquipo} D.F. Total

{idMovimiento} -> {idLaboratorioAnterior} D.F. Total

{idMovimiento} -> {idLaboratorioNuevo} D.F. Total

{idMovimiento} -> {cedulaUsuario} D.F. Total

{idMovimiento} -> {fechaMovimiento} D.F. Total

La relación se encuentra en 2FN porque todos los atributos no primos dependen totalmente de la clave primaria.

Tercera Forma Normal

Se analiza si algún atributo no primo determina a otro:

{idEquipo} -> {idLaboratorioAnterior}? No existe (un equipo se mueve varias veces y cada movimiento tiene su origen)

{idLaboratorioAnterior} -> {idLaboratorioNuevo}? No existe (desde un mismo laboratorio se puede mover a distintos destinos)

{idLaboratorioNuevo} -> {idLaboratorioAnterior}? No existe (a un mismo destino se llega desde distintos laboratorios)

{cedulaUsuario} -> {idEquipo}? No existe (un usuario mueve varios equipos)

Ningún atributo no primo determina a otro atributo no primo. La relación se encuentra en 3FN.

Resultado

MOVIMIENTO_EQUIPO (idMovimiento, idEquipo, idLaboratorioAnterior, idLaboratorioNuevo, cedulaUsuario, fechaMovimiento)

PK: idMovimiento

idEquipo es FK de EQUIPO(idEquipo)

idLaboratorioAnterior es FK de LABORATORIO(idLaboratorio)

idLaboratorioNuevo es FK de LABORATORIO(idLaboratorio)

cedulaUsuario es FK de USUARIO(cedula)

Relación INTERVENCION

INTERVENCION (idIntervencion, idEquipo, tipo, descripcion, cedulaTecnico, fechaIntervencion)

Clave Primaria: {idIntervencion}

Primera Forma Normal

Se encuentra en 1FN porque todos sus atributos son atómicos.

Segunda Forma Normal

Clave de un solo atributo ({idIntervencion}), por lo tanto no puede existir dependencia parcial. Se analiza:

{idIntervencion} -> {idEquipo} D.F. Total

{idIntervencion} -> {tipo} D.F. Total

{idIntervencion} -> {descripcion} D.F. Total

{idIntervencion} -> {cedulaTecnico} D.F. Total

{idIntervencion} -> {fechaIntervencion} D.F. Total

La relación se encuentra en 2FN porque todos los atributos no primos dependen totalmente de la clave primaria.

Tercera Forma Normal

Se analiza si algún atributo no primo determina a otro:

{tipo} -> {descripcion}? No existe (la descripción es texto libre propio de cada intervención)

{idEquipo} -> {cedulaTecnico}? No existe (un mismo equipo es intervenido por distintos técnicos)

{cedulaTecnico} -> {tipo}? No existe (un técnico realiza distintos tipos de intervención)

{tipo} -> {idEquipo}? No existe (un mismo tipo de intervención se aplica a muchos equipos)

Ningún atributo no primo determina a otro atributo no primo. La relación se encuentra en 3FN.

Resultado

INTERVENCION (idIntervencion, idEquipo, tipo, descripcion, cedulaTecnico, fechaIntervencion)

PK: idIntervencion

idEquipo es FK de EQUIPO(idEquipo)

cedulaTecnico es FK de TECNICO(cedula)

Relación REEMPLAZO

REEMPLAZO (idReemplazo, idEquipo, componente, descripcion, cedulaTecnico, fechaReemplazo)

Clave Primaria: {idReemplazo}

Primera Forma Normal

Se encuentra en 1FN porque todos sus atributos son atómicos.

Segunda Forma Normal

Clave de un solo atributo ({idReemplazo}), por lo tanto no puede existir dependencia parcial. Se analiza:

{idReemplazo} -> {idEquipo} D.F. Total

{idReemplazo} -> {componente} D.F. Total

{idReemplazo} -> {descripcion} D.F. Total

{idReemplazo} -> {cedulaTecnico} D.F. Total

{idReemplazo} -> {fechaReemplazo} D.F. Total

La relación se encuentra en 2FN porque todos los atributos no primos dependen totalmente de la clave primaria.

Tercera Forma Normal

Se analiza si algún atributo no primo determina a otro:

{componente} -> {descripcion}? No existe (la descripción es texto libre propio de cada reemplazo)

{idEquipo} -> {componente}? No existe (en un mismo equipo se reemplazan distintos componentes a lo largo del tiempo)

{componente} -> {idEquipo}? No existe (un mismo tipo de componente se reemplaza en muchos equipos)

{cedulaTecnico} -> {componente}? No existe (un técnico reemplaza distintos componentes)

Ningún atributo no primo determina a otro atributo no primo. La relación se encuentra en 3FN.

Resultado

REEMPLAZO (idReemplazo, idEquipo, componente, descripcion, cedulaTecnico, fechaReemplazo)

PK: idReemplazo

idEquipo es FK de EQUIPO(idEquipo)

cedulaTecnico es FK de TECNICO(cedula)

Patrones de diseño

Se logró identificar un patrón DAO en el código. El DAO es un patrón de diseño que separa la lógica de la base de datos.

Patrón de diseño	Que resuelve	Ubicación	Fragmentos
DAO (Data Access Objet)	Separa el SQL del sistema. Los controladores no conocen tablas y tampoco las consultas. Solo piden listar, obtener o crear.	app/modelo/DAO[todas las versiones].php	Figuras 1 y 2: constructor con PDO.
Front Controller	Evita tener archivos PHP por cada pantalla. Todas las solicitudes entran por un único punto. Donde se inicia sesión y se manejan los errores	public/index.php y app/rutas/api.php	Figura 3: tabla de endpoints.

MVC	Separa responsabilidades. Datos, lógica de la solicitud y presentación	Modelo: app/modelo Controlador: app/controlador Vista: app/vista/RespuestaJson.php Cliente: /public	Figura 4: un controlador que usa DAO y responde con la vista.

Figura 1

Codigo en php de constructor con inyección de PDO

```
class AccesoDatosUsuario {
    private PDO $conexion;

    public function __construct(PDO $conexion) {
        $this->conexion = $conexion;
    }
    // ...
}
```

Nota. Elaboración propia

Figura 2

Método de consulta con parámetros — de DAOEquipo::obtener().

```
public function obtener(string $idEquipo): ?array
{
```

```
    $sql = "
        SELECT e.idEquipo, e.idLaboratorio,
l.numeroLaboratorio AS laboratorio,
               e.marca, e.estado, e.disponibilidad,
e.informacion
        FROM EQUIPO AS e
        INNER JOIN LABORATORIO AS l ON l.idLaboratorio =
e.idLaboratorio
        WHERE e.idEquipo = :idEquipo
    ";


    $consulta = this -> conexion -> prepare(sql);
    $consulta->execute([":idEquipo" => $idEquipo]);


    $equipo = $consulta->fetch(PDO::FETCH_ASSOC);


    return $equipo === false ? null : $equipo;
}
```

Nota. Elaboración propia

Figura 3

Front Controller — de public/index.php y app/rutas/api.php.

```
 // public/index.php
$ruta = $_GET["ruta"];
require RUTA_RUTAS . "/api.php";


// app/rutas/api.php
$endpoints = [
```

```
    "equipos" => ["ControladorEquipo", "gestionar"],
    "tickets" => ["ControladorTicket", "gestionar"],
    // ...
];

[$nombreControlador, $metodoControlador] = endpoints[ruta];
require_once RUTA_CONTROLADOR . "/" . $nombreControlador .
".php";

$controlador = new $nombreControlador();
controlador ->metodoControlador();
```

Nota. Elaboración propia.

Figura 4

Controlador que usa el DAO y responde con la vista — de ControladorEquipo.

```
private function conectar(): void
{
    $conexion = Conexion::conectar();

    this -> dao = newDAOEquipo(conexion);
}

// al final de gestionar():
RespuestaJson::exito($resultado["datos"],
$resultado["mensaje"], $resultado["codigo"]);
```

Nota. Elaboración propia.

API

Propósito y responsabilidades

La API es la única vía de comunicación entre el cliente y el servidor. Sus responsabilidades:

- Recibe las solicitudes por un único punto: public/index.php?ruta=[nombre]
- Válida la sesión, el rol de un usuario y en las operaciones que modifican datos el token CSRF.
- Valida los datos que recibe y consulta o modifica la base de datos mediante los DAO
- Siempre responde en JSON, con un formato uniforme y su respectivo código HTTP.

Recursos y endpoints principales

Ruta	Métodos	Acceso
login	**POST**	**Público**
logout	**POST**	**Sesión iniciada**
sesion	**GET**	**Sesión iniciada**
idioma / traducciones	**POST / GET**	**Sin restricción por rol**
usuarios	**GET, POST, PUT, DELETE**	**Coordinador**
laboratorios, equipos	**GET, POST, PUT, DELETE**	**GET: coordinador, técnico y docente. El resto: coordinador y técnico**
prestamos	**GET, POST, PUT, DELETE**	**Coordinador y técnico**
tickets*	**GET, POST, PUT, DELETE**	**Coordinador, técnico y docente (según el método)**
solicitudes*	**GET, POST, PUT, DELETE**	**Docente y coordinador/técnico (según el método)**

diagnosticos, reparaciones, soluciones	**GET, POST, PUT, DELETE**	**Alta: técnico. El resto: coordinador y técnico**
intervenciones, reemplazos	**POST**	**Técnico**
historialEquipo	**GET**	**Coordinador y técnico**
dashboard	**GET**	**Coordinador y técnico**

Estructura de respuestas y códigos de estado

Todas las respuestas manejan la misma estructura (se define en RespuestaJson).

{ "status": "success" | "error", "message": "texto", "data": ... }

En los errores data siempre es null.

Códigos que usa el sistema:

Código	Cuando
200	Consulta o modificación correcta.
201	Recurso creado.
400	Datos inválidos, filtro no permitido o referencia a algo que no existe.
401	Sesión no iniciada o credenciales incorrectas.
403	Rol incorrecto o token CSRF no válido.
404	Recurso no existente o ruta no encontrada.
405	Método HTTP no permitido
409	Registro duplicado o no se puede eliminar por tener datos que se comparten
500	Error interno (Se guarda y se entrega un mensaje genérico al cliente).

Contrato JSON

- **Solicitud**. Las operaciones con cuerpo mandan “Content-Type: application/json” y los datos en JSON. Todo lo que modifica datos agregan en la parte de arriba X-CSRF-Token. El servidor lee todo con “Solicitud::obtenerDatos”.
- **Respuesta**. Siempre application/json; charset=utf-8 con el formato antes mencionado.
- **Mensajes**. Se devuelven en el idioma elegido en la sesión.

Ejemplos

Elemento	**Ej 1**	**Ej 2**
Operación	Consultar un equipo por su id	Registrar un equipo
Método y ruta	GET /public/index.php?ruta=equipos&id=PC-01	POST /public/index.php?ruta=equipos
Parámetro / Cuerpo	id:texto, máximo 10 caracteres	JSON con idEquipo, idLaboratorio, marca, estado, disponibilidad, informacion
Acceso	Coordinador, técnico o docente	Coordinador o técnico con X-CSRF-Token
Éxito	200 OK	201 Created
Error	404 Not Found	409 Conflict (ya existe la id)

Respuesta satisfactoria del ejemplo 1:


```
{
  "status": "success",
  "message": "Equipo encontrado.",
  "data": {
```

```
    "idEquipo": "PC-01",
    "idLaboratorio": "LAB001",
    "laboratorio": "Lab 1",
    "marca": "Dell",
    "estado": "Funcionando",
    "disponibilidad": "Disponible",
    "informacion": "Equipo de escritorio"
  }
}
```


Respuesta de error


```
{
  "status": "error",
  "message": "No existe un equipo con el identificador PC-99.",
  "data": null
}
```


API FETCH

Todo cliente usa la función única “llamarapi”.

- Se arman las URL con “?ruta=” y con parámetros no vacíos.
- Agrega X-CSRF-Token (en sessionStorage) solo cuando el método no es GET.
- Envía el cuerpo como JSON.
- Convierte los errores HTTP en excepciones, si hay un 401 vuelve al login.

Criterios REST

Criterio	Estado
Recursos identificados con sustantivos (equipos, tickets)	Se cumple
Verbos HTTP con su significado (GET lee, POST crea, PUT modifica, DELETE elimina)	Se cumple
Códigos de estado adecuados (201 al crear, 404, 409…)	Se cumple
Representación en JSON y formato uniforme	Se cumple
GET sin efectos secundarios	Se cumple
Rutas jerárquicas	No se cumple
Sin estado (stateless)	No se cumple

Acceso al repositorio

Enlace al repositorio:

https://github.com/augustocrafter22-lab/ProyectoFinal/tree/main

El repositorio es de visibilidad privada por lo que se tendrá que otorgar el acceso para poder visualizarlo.

Anexos

Historial de commits:

Historial completo:

https://github.com/augustocrafter22-lab/ProyectoFinal/commits/main

Issues seleccionados:

Issue	**Hash**	**Mensaje**
#3 Registrar equipos RF1	ad2a4d0	feat(Equipos): Agrega funcionalidad de registrar equipos en una tabla utilizando el DOM resolves #3
#38 Registrar diagnósticos técnicos RF36	a1d8e9a	Registrar Diagnostico
#21 Crear tickets de incidencias RF19	b939338	Crear TicketRF19
#48 Iniciar sesión RF46	b81435d	fix(login): Se soluciono que cuando salia el error de contraseña incorrecta empujaba otras etiquetas. Resolves #48
#154 alta de usuarios y roles	36c0536	Feat Implementacion de usuarios +2 roles
#164 Cargar listado de equipos desde la base de datos	ce15e12	feat:implementacion de la base de datos a equipos
#209 Interfaz bilingüe	4dc145f	Interfaz Bilingüe
#210 Módulo de préstamos	3210ab6	feat: módulo de préstamos y devoluciones de equipos
#213 Orden de parámetros en crearUsuario	4bcf211	fix: corregir alta de usuarios

Listado de issues:

3 Registrar equipos RF1

4 Asignar marca a cada equipo. RF2

5 Clasificar equipos por estado RF3

6 Clasificar equipos por tipo RF4

7 Clasificar equipos por ubicación RF5

8 Modificar información de un equipo RF6

9 Consultar datos generales de un equipo RF7

10 Buscar equipos por número de serie RF8

11 Filtrar equipos según su estado RF9

12 Filtrar equipos según su ubicación RF10

13 Registrar cambios de estado de un equipo RF11

14 Registrar cambios de ubicación de un equipo RF12

15 Consultar movimientos de un equipo RF13

16 Consultar historial de estados de un equipo RF14

17 Mostrar disponibilidad actual de los equipos RF15

18 Mostrar equipos fuera de servicio RF16

19 Consultar estado actual de un equipo RF17

20 Mostrar equipos disponibles para su uso RF18

21 Crear tickets de incidencias RF19

22	Asociar un ticket a un equipo RF20
23	Ingresar descripción para un ticket RF21
24	Registrar fecha de creación de un ticket RF22
25	Consultar tickets registrados RF23
26	Consultar laboratorio asociado a un equipo RF24
27	Consultar incidencias asociadas a un equipo RF25
28	Filtrar incidencias según prioridad RF26
29	Visualizar tickets pendientes RF27
30	Visualizar tickets en proceso RF28
31	Visualizar tickets resueltos RF29
32	Actualizar estado de un ticket RF30
34	Buscar tickets mediante palabras clave RF32
35	Filtrar tickets según su estado RF33
36	Filtrar tickets por rango de fechas RF34
37	El sistema deberá registrar la fecha de finalización de un ticket. RF35
38	Registrar diagnósticos técnicos RF36
39	Registrar soluciones aplicadas a una incidencia RF37
40	Asociar diagnóstico a un ticket RF38
41	Asociar solución a un ticket RF39
42	Consultar diagnósticos registrados RF40
43	Modificar diagnósticos registrados RF41
44	Almacenar historial de reparaciones de equipos RF42
45	Almacenar intervenciones realizadas sobre equipos RF43

46 Almacenar reemplazos de componentes RF44

47 Consultar historial técnico de un equipo RF45

48 Iniciar sesión mediante usuario y contraseña RF46

49 Cerrar sesión RF47

50 Mostrar estadísticas de incidencias registradas RF48

51 Mostrar estadísticas de incidencias resueltas RF49

52 Mostrar tiempos de resolución de incidencias RF50

53 Mostrar estadísticas de uso de laboratorios RF51

54 Solicitar preparación de laboratorios RF52

55 Solicitar instalación de software RF53

56 Registrar versión requerida del software solicitado RF54

57 Registrar requerimientos adicionales para la instalación de software RF55

58 Definir restricciones de uso para estudiantes RF56

59 Consultar solicitudes de servicio registradas RF57

60 Filtrar solicitudes de servicio según su tipo RF58

62 Registrar turno al crear ticket RF59 NUEVO RF RF25

63 Registrar taller asociado al ticket RF60 NUEVO RF26

64 Registrar docente creador del ticket RF61 NUEVO RF27

65 Registrar grupo asociado al ticket RF62 NUEVO RF28

85 Crear barra de navegacion para la pagina

140 Rediseño y unificación estética de los componentes HTML de equipos

144 Eliminar archivos sin uso

152 Configurar rutas del proyecto

153 Adaptar formulario de alta de empleados

154 Implementar acceso a datos para registrar usuarios

155 Registrar rol y controlar transacción del alta

156 Integrar alta de empleados con el panel de administración

157 Crear modelo de usuario para el login

158 Redirigir usuario según su rol

159 Revisar DER y normalización

160 Crear archivos SQL

162 Cargar listado de tickets desde la base de datos

163 Cargar listado de diagnósticos desde la base de datos

164 Cargar listado de equipos desde la base de datos

165 Adaptar JavaScript de tickets a la nueva carga mediante PHP

166 Adaptar JavaScript de diagnósticos a la base de datos

167 Asociar diagnóstico a un ticket

168 Registrar solución asociada a un diagnóstico

169 Registrar reparación realizada sobre un equipo

170 Implementar historial de incidencias por equipo (CON BASE DE DATOS)

171 Actualizar estado del ticket durante su resolución

172 Registrar fecha de finalización del ticket (CON BASE DE DATOS)

173 Implementar dashboard de tickets (FUNCIONAL)

174 Asociar tickets con equipos (BDD)

175 Adaptar filtros de tickets

176 Adaptar filtros de diagnósticos

177 Adaptar filtros de equipos

200 Contrato JSON estándar para la API

201 API REST de Usuarios

202 API REST de Equipos

203 API REST de Tickets

204 API REST de Diagnósticos, Soluciones y Reparaciones

205 API REST de Laboratorios y Solicitudes de laboratorio

206 Consumo de la API con Fetch en todas las pantallas

207 Consultas preparadas en todo el sistema

208 Documento de persistencia de la tercera entrega

209 Interfaz bilingüe (español e inglés)

210 Módulo de préstamos y devoluciones de equipos

211 Guardar movimientos, historial de estados, intervenciones y reemplazos

212 Acceso del Coordinador al dashboard

213 Orden de parámetros en crearUsuario (CORRECCION PROFE)

214 Baja de tickets (CORRECCION)

215 Manejo de excepciones sin exponer información interna (CORRECCION)

216 Validación uniforme de campos en el servidor (CORRECCION)

217 Claves foráneas en la tabla de tickets (CORRECCION)

218 Normalización completa del modelo de datos (CORRECCION)

219 Separar responsabilidades en los controladores (CORRECCION)

220 Documentar los patrones de diseño con justificación (CORRECCION)

221 Integración final del sistema

Bibliografía utilizada

Bibliografía utilizada

Bootstrap Icons. https://icons.getbootstrap.com/

Figma Help Center. https://help.figma.com/

GitHub Docs. https://docs.github.com/

Git Documentation. https://git-scm.com/doc

Visual Studio Code Docs. https://code.visualstudio.com/docs

Herramientas de investigación

Para resolver dudas puntuales durante el desarrollo, el equipo utilizó las siguientes herramientas de investigación:

-Google

-Documentación oficial de las herramientas utilizadas

-Asistentes de Inteligencia Artificial para facilitar búsquedas